

Simulation Overview

The provided Python code models the growth of neurites in a confined 3D space, capturing several key biological processes: dynamic pathfinding, branching, and synaptogenesis. The simulation progresses in discrete time steps, iteratively updating the state of multiple growing neurites. The core of the model is its ability to integrate stochastic (random walk) and deterministic (guidance from a vector field and boundary reflection) processes to generate complex, bio-plausible network topologies.

Core Algorithmic Components

The simulation is built around the **Core_growth** function, which orchestrates a primary while loop that continues as long as there are neurites with a remaining length budget. Within this loop, the following functions are called in sequence:

- 1. Neurite Pathfinding:** The movement of a neurite tip at each step is governed by its current location and its direction, defined by the azimuthal (ϕ) and polar (θ) angles. The new position ($x_{new}, y_{new}, z_{new}$) is calculated from the previous position ($x_{old}, y_{old}, z_{old}$) and a fixed step size DI using spherical-to-Cartesian conversion:
 - $x_{new} = x_{old} + DI \cdot \cos(\phi) \cdot \sin(\theta)$
 - $y_{new} = y_{old} + DI \cdot \sin(\phi) \cdot \sin(\theta)$
 - $z_{new} = z_{old} + DI \cdot \cos(\theta)$ This model combines a basic random walk with external influences. The angles ϕ and θ are continuously perturbed by a random variable drawn from a normal distribution with standard deviations ϕ_sd and θ_sd , respectively. This introduces a realistic level of Brownian-like motion into the growth process.
- 2. Boundary Interaction (Check_borders):** This function implements a hard boundary condition for the simulation space. When a neurite's calculated next position exceeds the width, depth, or height of the culture, its trajectory is modified. The function uses a simple "reflection" logic:
 - If the neurite crosses an x or y boundary, the azimuthal angle (ϕ) is adjusted.
 - If it crosses a z boundary, the polar angle (θ) is adjusted. To prevent a neurite from becoming trapped at a boundary, the magnitude of the angular adjustment increases with each consecutive boundary collision. This vectorized implementation efficiently handles multiple neurites in parallel.
- 3. Vector Field Guidance:** If the `Vector_field_check` flag is enabled, the simulation introduces an external guidance cue. This is implemented by using a pre-computed vector field. At each step, the neurite's position is used to query a `ckdTree` (`Vector_field_tree`) to find the nearest vector in the field. This vector then influences the neurite's current direction. This mechanism can be used to model phenomena like **chemotaxis**, where neurites are guided by molecular gradients.
- 4. Synaptogenesis (Check_synapses):** This function, though not fully detailed in the provided code, is responsible for detecting and establishing synaptic connections. A neurite can only form a synapse if its tip enters a "synaptic detection zone" defined by the `r_dendrite` around a target neuron's soma. The probability of connection is determined by two factors:
 - **Distance-based Probability:** The `get_dendrite_prob` function calculates the likelihood of a neurite hitting a dendritic process based on the distance from the soma. This function uses a scaled Chi-squared distribution to model the non-uniform density of dendritic

arborization, which is known to peak at a certain distance from the soma in hiPSC neurons. The probability is calculated as the ratio of the "filled volume" (representing dendrites) to the total volume of a spherical shell.

- **Molecular Specificity:** The connection probability is further modulated by the `alpha_values`, a set of pre-defined bias terms that favor or disfavor specific connection types (E-E, E-I, I-E/I), reflecting biological reality.
5. **Branching:** This is a core stochastic process in the simulation. At each time step, for every growing neurite, a random number is generated and compared to the user-defined `P_b` (branching probability). If a branch event is triggered, a new neurite is created.
- **New Branch Length:** The length of the new branch is not fixed but is determined by sampling from a **Rayleigh distribution**. This is a biologically informed choice, as the Rayleigh distribution models the magnitude of a 2D random walk, which is a good analogy for the growth of a new branch. The length is then scaled to a value between 0 and the remaining length of the parent neurite.
 - **New Branch Direction:** The new branch inherits its initial position from the parent neurite's tip. Its initial direction is determined by adding a random angle to the parent neurite's angles, ensuring that the new branch extends in a different direction.

Key Computational and Biological Concepts

- **Vectorization:** The code is highly optimized for performance by using vectorized NumPy operations wherever possible. Instead of looping through each neurite individually, it processes all active neurites simultaneously, which is crucial for scaling the simulation to thousands of neurites.
- **Coordinate Systems:** The simulation uses a hybrid coordinate system. Neurite pathfinding is calculated using spherical coordinates (ϕ and θ) for intuitive directional control, and the results are then converted to Cartesian coordinates for spatial representation and boundary checking.
- **Data Structures:** The use of a **cKDTree** is a key computational detail. This data structure allows for highly efficient nearest-neighbor searches in 3D space, which is essential for quickly finding the nearest vector field point or target neuron without a brute-force search.
- **Bio-Plausibility:** The model is grounded in real biological data, particularly from the cited papers on hiPSC neuron morphology. Parameters like `Branch_d` (neurite diameter) and the use of a chi-squared distribution to model dendritic density lend a level of biological realism to the simulation's rules.

1. Explicit Parameters: The Experimental Design and Controls

These variables are the primary knobs you can turn to control the simulation. They define the physical, biological, and computational constraints of your experiment.

- **Total_length**: A NumPy array of size N (number of neurites), where each element is a float representing the total growth budget for a single neurite, in micrometers. Growth ceases for a given neurite when its length has been exhausted. This allows for modeling the heterogeneity of neurite lengths seen in biological systems.
- **Xi, Yi, Zi**: These are three 2D NumPy arrays, each of shape (N,K), where N is the number of neurites and K is the maximum number of segments. They act as a **spatiotemporal record** of each neurite's path. During the simulation, the n-th column of these arrays is filled with the coordinates of the neurites at the n-th growth step.
- **DI**: The constant step size, or segment length, in micrometers. It represents a discrete unit of growth. The choice of DI is a classic **trade-off between accuracy and speed**: a smaller DI provides a more granular, geometrically accurate path but significantly increases the number of iterations required to reach a neurite's Total_length, leading to longer runtimes.
- **width, depth, height**: Floats that define the dimensions of the rectangular simulation space in micrometers. These represent the boundaries of your culture, and they're crucial for confining neurite growth. The simulation operates within a Cartesian coordinate system, with these parameters setting the upper bounds of the x, y, and z axes (assuming the origin is at [0, 0, 0]).
- **X, Y, Z**: NumPy arrays storing the 3D coordinates of all neuron somata in the culture. These are the **target locations** for synaptogenesis.
- **Connections**: An array of lists that functions as an **adjacency list** of the developing neural network. Connections[i] is a list of indices of all neurons that neuron i has established a synapse with. This is a vital data structure for managing network topology and preventing a neurite from forming redundant synapses with an already-connected neuron.
- **alpha_values**: A 3-element list of floats that provides a **synaptic probability bias**. These values are multiplied with a calculated base probability to fine-tune the likelihood of different connection types, allowing you to model **synaptic specificity**. The indices correspond to:
 1. alpha_values[0]: Excitatory-to-Excitatory (E-E) connections.
 2. alpha_values[1]: Excitatory-to-Inhibitory (E-I) connections.
 3. alpha_values[2]: Inhibitory-to-Excitatory/Inhibitory (I-E/I) connections.
- **r_dendrite**: A NumPy array where each element is the radius (in micrometers) of a neuron's perisomatic dendritic arborization. This value defines the **sphere of influence** around a soma where a neurite can potentially form a synapse. It's a critical parameter for modeling the spatial constraint of synaptogenesis.
- **Neuron_of_Neurites_idx**: A key bookkeeping array that maps each growing neurite to its parent neuron's index. This is essential for assigning biological properties (like neuron type) to each neurite.

- **Ne:** An integer indicating the number of excitatory neurons. It serves as a threshold to classify neurons as either excitatory or inhibitory (neurons < Ne are excitatory, neurons >= Ne are inhibitory).
- **P_b:** The **probability of branching** (a float between 0 and 1) at each growth step. This parameter is the primary control for the neurite's **morphological complexity**. A higher P_b increases the likelihood of a neurite splitting, resulting in a more intricate, tree-like structure.
- **Branch_TH:** A length threshold (in micrometers). If a newly generated branch is shorter than this value, it's discarded. This feature acts as a filter for biologically insignificant branches, which are often non-functional.
- **phi_sd, theta_sd:** Floats that represent the standard deviations for the normal distributions used to introduce **stochasticity** into the neurite's direction. These parameters model the inherent randomness of neurite pathfinding. Higher values lead to more winding, erratic paths, while lower values produce straighter, more direct growth.
- **Ns:** An integer representing the maximum number of segments that any single neurite can have. This defines the column dimension of the Xi, Yi, Zi arrays, effectively setting a hard limit on the length of any neurite, regardless of its Total_length budget.
- **N_neurites:** The number of neurites in a specific processing batch.
- **Vector_Field, Vector_field_surrogate, Vector_field_tree, over_factor:** These parameters are used to implement **chemoattractive guidance**. Vector_Field stores pre-computed vector gradients, and Vector_field_tree (a cKDTree) allows for a fast nearest-neighbor search to find the vector closest to a neurite tip. If **Vector_field_check** is True, the neurite's direction is influenced by this vector, simulating guidance by a global molecular gradient.
- **Border_check, Vector_field_check:** Boolean flags that act as **on/off switches** for the border confinement and vector field guidance features, allowing for controlled experiments.

2. Implicit Parameters: The Algorithmic Substructure

These variables are not set by the user but are hard-coded or dynamically generated to implement the simulation's rules.

- **A:** The **Adjacency Matrix**, a NumPy array of zeros that is a secondary, often final, representation of the network connectivity. A value of 1 at A[i, j] signifies an established connection from neuron i to neuron j.
- **phi, theta:** These NumPy arrays store the instantaneous **azimuthal (ϕ) and polar (θ) angles** for each neurite. They are updated at every step based on random noise, border reflections, and vector field influence.
- **sd, debug_angle, p_add, max_iter:** These are internal parameters of the Check_borders function. sd and debug_angle define the initial magnitude of the angular change when a neurite hits a border. p_add is a small increment that increases the angular change with each failed reflection attempt, preventing a neurite from getting "stuck" at the boundary. max_iter serves as a failsafe to prevent infinite loops.

- **Branch_d:** This float, found within `get_dendrite_prob`, is a hard-coded neurite diameter based on experimental data. It changes based on the distance from the soma, reflecting the biological reality of neurite tapering. It is set to 2.1 μm for primary neurites and 1.51 μm for secondary neurites.
- **r_0, r_1, dx, map_magnitude:** These are parameters used in `get_dendrite_prob` to model the dendritic field as a chi-squared distribution. They define the inner (`r_0`) and outer (`r_1`) radii of the spherical shell, the step size for numerical integration (`dx`), and the mapping scale (`map_magnitude`). They are used to calculate the probability of a growing neurite crossing a dendrite at a specific distance from the soma.

3. Interrelationships and Biological Implications

The simulation's power lies in the intricate interplay of these parameters, which allows emergent network properties to arise from simple, localized growth rules.

- **Growth and Morphology:** The overall morphology of a neurite is a direct consequence of the interplay between **Total_length**, **DI**, **P_b**, and the angular stochasticity (`phi_sd`, `theta_sd`). For a given **Total_length**, a smaller **DI** means more steps, and thus more opportunities for branching and angular changes. A high **P_b** will lead to a dense, fractal-like structure, while a low **P_b** will result in a simpler, less branched neurite.
- **Guided vs. Random Growth:** The simulation can model two fundamental modes of neurite growth. When **Vector_field_check** is `True`, growth is **directed** by a global external signal. When it's `False`, neurites engage in a **stochastic search process** driven by internal randomness. The values of `phi_sd` and `theta_sd` tune the degree of this randomness, affecting how much the neurite explores its environment versus moving along a straight path.
- **Synaptic Specificity:** Synapse formation is not a simple contact-based event but a probabilistic process.
 1. **Spatial Proximity:** A neurite must first enter the **r_dendrite** sphere of influence.
 2. **Dendritic Density:** The `get_dendrite_prob` function then uses the neurite's distance from the soma to calculate a base probability of hitting a dendrite. This is a highly detailed biological model based on the experimental observation that dendritic density is non-uniform and peaks at a specific distance from the soma.
 3. **Molecular Identity:** Finally, the **alpha_values** bias this probability based on the known molecular identities of the pre- and post-synaptic neurons, reflecting that certain connections (e.g., E-E) are more favored than others.
- **Computational Efficiency:** The design of the code, with its vectorized NumPy operations and the use of a cKDTree for fast spatial queries, is optimized for large-scale simulations. This is crucial for studying how local growth rules at the individual neurite level give rise to large-scale network topologies, a key question in modern neuroscience.